

You need to use 'o' prudently since it gives everyone read and write access. The permitted permissions that you can give to a file are explained below:

- r for read
- w for write
- e for execute

The use of the -R key is important. If there are many subfolders and files in the SHA directory and you wish to apply for the permissions from the parent folder to all the dependent subfolders then you need to use the -R (recursion) key.

Dealing with file permissions

Changing the ownership of a file or folder is easy. Let's say David transited a folder for Mary in the SHARE library - but David still has ownership. This can be changed with simple commands like:

- `sudo chown -R Mary /DATA/SHARE`

Understanding the umask command with examples

In GNU / Linux systems, each file or folder has permissions. There are three types of permissions (those allowed for each file type, including directories).

- (R) Read access
- (W) Write permission
- (E) Execute

Permissions are defined for three types of users.

- (U) file owner
- (G) A group to which the owner belongs
- (O) All other users

umask (User Mask) is a POSIX environment command and function that defines the file creation mask for the current process, which determines the permissions of files and directories created by the process. The process may change the file status creation mask using umask, and the subsystem will obtain the new value. You can use umask to set new file permissions that the process creates. The user mask contains octal permission values that are assigned to all new files. To calculate the umask value, the authorisation value from which you have to subtract is 666 for files and 777 for directories. The value you get then has to be used with the umask command.

For instance, there is a need for changing the default file mode to 664 (rw-rw-r-). On subtracting between 666 and 664, you get 002. This is the value that will be used as an argument to the umask command, as shown below:

- `Umask 002`